

E-Commerce Data Analysis using Python

This project analyzes e-commerce sales data using Python to generate meaningful business insights. It includes data generation, cleaning, exploratory data analysis, and visualization to understand trends in sales, customer behavior, and revenue patterns.

Tools & Technologies: Python, Pandas, NumPy, Matplotlib, Seaborn

Project Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os

# #####
# 0. Setup
# #####
np.random.seed(42)
os.makedirs("outputs", exist_ok=True)

sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams["figure.dpi"] = 130

# #####
# 1. Generate Realistic Sample Dataset
# #####
def generate_data(n=500):
    """Create a synthetic e-commerce orders dataset."""
    categories = ["Electronics", "Clothing", "Books", "Home & Kitchen", "Sports"]
    regions = ["North", "South", "East", "West"]
    payment_methods = ["Credit Card", "UPI", "Net Banking", "Cash on Delivery"]

    dates = pd.date_range(start="2023-01-01", end="2023-12-31", periods=n)

    category_col = np.random.choice(categories, n)
    price_map = {
        "Electronics": (500, 50000),
        "Clothing": (300, 5000),
        "Books": (100, 2000),
        "Home & Kitchen": (200, 15000),
        "Sports": (400, 20000),
    }

    prices = np.array([np.random.randint(*price_map[c]) for c in category_col])
    quantities = np.random.randint(1, 6, n)
    discounts = np.random.choice([0, 5, 10, 15, 20], n)
    revenue = prices * quantities * (1 - discounts / 100)

    df = pd.DataFrame({
        "order_id": range(1001, 1001 + n),
        "date": dates,
        "category": category_col,
        "region": np.random.choice(regions, n),
        "payment_method": np.random.choice(payment_methods, n),
        "price": prices,
        "quantity": quantities,
        "discount_pct": discounts,
        "revenue": revenue.round(2),
        "customer_rating": np.random.randint(1, 6, n),
    })

    # Inject a few nulls to practise cleaning
    df.loc[np.random.choice(df.index, 15), "customer_rating"] = np.nan
    return df

# #####
```

[illegible]

[illegible]

This project demonstrates how data analysis can be used to extract actionable insights and support business decision-making using Python.